
IFM 3.1: Compilerbau (Winter 2026/27)

BC George & Carsten Gips (HSBI)

29. September 2026

Inhaltsverzeichnis

I. Syllabus	1
1. Syllabus	2
1.1. Kursbeschreibung	2
1.2. Überblick Modulinhalte	2
1.3. Team	3
1.4. Kursformat	3
1.5. Fahrplan	3
1.6. Prüfungsform, Note und Credits	4
1.6.1. Studienleistung: "Portfolio" - Kriterien je Person:	4
1.6.2. Gesamtnote: Parcoursprüfung	5
1.6.3. Hinweise	5
1.7. Materialien	5
1.8. Förderungen und Kooperationen	6
1.8.1. Kooperation mit University of Alberta, Edmonton (Kanada)	6
1.9. LICENSE	6
II. Vorlesungsunterlagen	7
2. Überblick	8
2.1. Struktur eines Compilers	8
2.1.1. Sprachen verstehen, Texte transformieren	8
2.1.2. Compiler: Big Picture	9
2.1.3. Lexikalische Analyse: Wörter ("Token") erkennen	11
2.1.4. Syntaxanalyse: Sätze erkennen	11
2.1.5. Vorschau: Parser implementieren	12
2.1.6. Semantische Analyse: Bedeutung erkennen	13
2.1.7. Zwischencode generieren	13
2.1.8. Code optimieren	14
2.1.9. Code generieren	14
2.1.10. Probleme	14
2.1.11. Wrap-Up	15
2.2. Bandbreite der Programmiersprachen	15
2.2.1. 99 Bottles of Beer	15
2.2.2. Imperativ, Hardwarenah: C	16

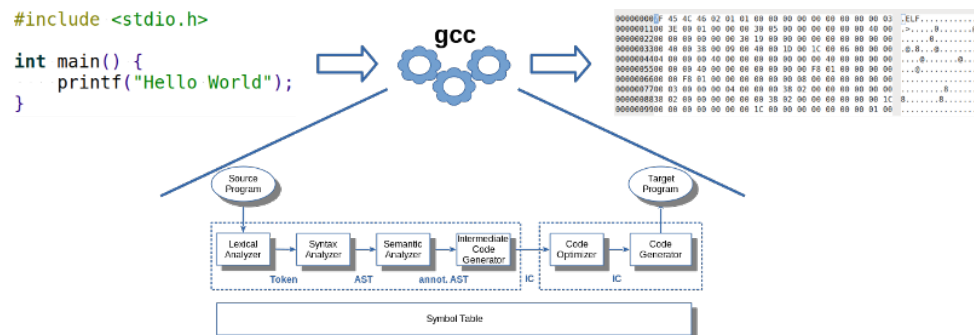
2.2.3.	Imperativ, Objektorientiert: Java	16
2.2.4.	Logisch: Prolog	17
2.2.5.	Funktional: Haskell	18
2.2.6.	Brainfuck	19
2.2.7.	Programmiersprache Lox	19
2.2.8.	Wrap-Up	20
2.3.	Anwendungen	20
2.3.1.	Anwendung: Compiler	21
2.3.2.	Anwendung: Interpreter	21
2.3.3.	Anwendung: Virtuelle Maschinen	21
2.3.4.	Anwendung: C-Toolchain	22
2.3.5.	Anwendung: C++-Compiler	22
2.3.6.	Anwendung: Bugfinder	22
2.3.7.	Anwendung: Pandoc	23
2.3.8.	Was bringt mir das?	24
2.3.9.	Wrap-Up	26
3.	Reguläre Sprachen, kontextfreie Grammatiken und Sprachen, lexikalische und syntaktische Analyse	28
3.1.	Reguläre Sprachen, Ausdrucksstärke (Teil 1)	28
3.1.1.	Motivation	28
3.1.2.	Endliche Automaten	28
3.1.3.	Reguläre Sprachen	31
3.1.4.	Wrap-Up	33
3.2.	Reguläre Sprachen, Ausdrucksstärke (Teil 2)	33
3.2.1.	Wiederholung	33
3.2.2.	Motivation	34
3.2.3.	Lexer	34
3.2.4.	Wrap-Up	35
3.3.	CFG	35
3.3.1.	Wiederholung	36
3.3.2.	Motivation	36
3.3.3.	Kontextfreie Grammatiken und Sprachen	38
3.3.4.	Syntaxanalyse	38
3.3.5.	Wrap-Up	39
3.4.	LL-Parser	40
3.4.1.	Wiederholung	40
3.4.2.	Motivation	40
3.4.3.	Syntaxanalyse	40
3.4.4.	LL(k)-Grammatiken	41
3.4.5.	Wrap-Up	44
4.	Sprache L-Int: Integer, Addition, Subtraktion	45
4.0.1.	Teil 1:	45
4.0.2.	Teil 2:	45

5. Sprache L-Expr: erweiterte Ausdrücke, Vorrangregeln	46
5.0.1. Inhalte	46
6. Sprache L-Var: Variablen, Statments, nested Scopes	47
6.0.1. Inhalt	47
7. Sprache L-If: Datentyp Boolean, Vergleiche, Kontrollstrukturen (if/else, while)	48
7.0.1. Inhalte	48
8. Sprache L-Fun: Funktionen (Definition, Aufruf)	49
8.0.1. Inhalte	49
9. Sprache L-Class: Klassen (Felder, Methoden, Objekte)	50
9.0.1. Inhalte	50
10. Sprache L-Inherit: Einfachvererbung und dynamischer Dispatch	51
10.0.1. Inhalte	51
11. Sprache L-Self: native Klassen (inkl. Vererbung)	52
11.0.1. Inhalte	52
 III. Praktikum	 53
12. Blatt 01: Reguläre Sprachen	55
12.1. A1.1: Sprachen von regulären Ausdrücken (1P)	55
12.2. A1.2: Bezeichner in Programmiersprachen (3P)	55
12.3. A1.3: Gleitkommazahlen in Programmiersprachen (2P)	55
12.4. A1.4: Mailadressen? (1P)	56
12.5. A1.5: Der zweitletzte Buchstabe (1P)	56
12.6. A1.6: Sprache einer regulären Grammatik (2P)	56
13. Blatt 02: CFG	57
13.1. A2.1: PDA (3P)	57
13.2. A2.2: Akzeptierte Sprache (2P)	57
13.3. A2.3: Kontextfreie Sprache (2P)	58
13.4. A2.4: Kontextfreie Grammatik (3P)	58

Teil I.

Syllabus

1. Syllabus



1.1. Kursbeschreibung

Der Compiler ist das wichtigste Werkzeug in der Informatik. In der Königsdisziplin der Informatik schließt sich der Kreis, hier kommen die unterschiedlichen Algorithmen und Datenstrukturen und Programmiersprachenkonzepte zur Anwendung.

In diesem Modul geht es um ein grundlegendes Verständnis für die wichtigsten Konzepte im Compilerbau. Wir schauen uns dazu relevante aktuelle Tools und Frameworks an und setzen diese bei der Erstellung eines kleinen Compiler-Frontends für C++ ein.

1.2. Überblick Modulinhalte

1. Lexikalische Analyse: Scanner/Lexer
 - Reguläre Sprachen
 - Generierung mit ANTLR
2. Syntaxanalyse: Parser
 - Kontextfreie Grammatiken (CFG)
 - LL-Parser (Recursive-Descent-Parser)
 - Generierung mit ANTLR
3. Semantische Analyse: Symboltabellen, Name-Resolving, Type-Checking
 - Namen und Scopes
 - Typen, Klassen, Polymorphie

4. Interpreter: AST-Traversierung
5. C++ als zu verarbeitende Programmiersprache

1.3. Team

- [BC George](#)
- [Carsten Gips](#) (Sprechstunde nach Vereinbarung)
- Alesia Herbertz, Vivien Traue, Jonathan Hauer (Tutor:innen)

1.4. Kursformat

Vorlesung (2 SWS)	Praktikum (2 SWS)
Mo, 15:45 - 17:15 Uhr (Zoom)	G1: Mi, 11:30 - 13:00 Uhr (Zoom)
	G2: Mi, 14:00 - 15:30 Uhr (Zoom)
	G3: Mi, 11:30 - 13:00 Uhr (Präsenz B40)
	G4: Mi, 14:00 - 15:30 Uhr (Präsenz B40)

Durchführung der Vorlesung als *Flipped Classroom* (Carsten) bzw. als *reguläre Vorlesung* (BC). **Zugangsdaten Zoom siehe [ILIAS](#).**

1.5. Fahrplan

Monat	Woche vom	Vorlesung (Mo)	Praktikum (Mi)	Edmonton/Minden-Meetings
Oktober	12.10.	Orga Überblick Sprachen Anwendungen	-	
	19.10.	Reguläre Sprachen 1	B01	
	26.10.	Reguläre Sprachen 2 CFG	B02	
November	02.11.	LL-Parser (Theorie)	-	Di, 03.11., 17:00 - 18:00 Uhr (online): ANTLR + Live-Coding
	09.11.	L-Int (Teil 1)	Station 1	
	16.11.	L-Int (Teil 2)	C-Int	
	23.11.	L-Expr	C-Expr	

Monat	Woche vom	Vorlesung (Mo)	Praktikum (Mi)	Edmonton/Minden-Meetings
Dezember	30.11.	L-Var	Station 2	Mo, 30.11., 17:00 - 18:00 Uhr (online): Minden Presentations
	07.12.	L-If	C-Var, C-If	Mo, 07.12., 17:00 - 18:00 Uhr (online): Edmonton Presentations
	14.12.	L-Fun	C-Fun	
	21.12.	Weihnachtspause	-	
	28.12.	Weihnachtspause	-	
Januar	04.01.	L-Class	Station 3	
	11.01.	L-Inherit	C-Class, C-Inherit	
	18.01.	L-Self	C-Self, Snake	
	25.01.	Rückblick	Projektvorstellung (Video)	

1.6. Prüfungsform, Note und Credits

Parcoursprüfung plus Studienleistung (Portfolio), 5 ECTS

1.6.1. Studienleistung: “Portfolio” - Kriterien je Person:

- Teilnahme an mind. zwei Edmonton/Minden-Terminen mit aktiver Beteiligung und Abgabe eines ausreichenden Post Mortems (pro Meeting, je Person)
 - Termin 1: Dienstag, 03.11., 17:00 - 18:00 Uhr (online)
 - Termin 2: Montag, 30.11., 17:00 - 18:00 Uhr (online)
 - Termin 3: Montag, 07.12., 17:00 - 18:00 Uhr (online)
 - Abgabe der Post Mortems zu den Edmonton-Meetings jeweils bis Montag 09:00 Uhr in der Folgewoche im [ILIAS](#)
- Abschluss-Video-Vortrag zum erfolgreich bearbeiteten Snake-Mini-Projekt am Semesterende
 - Termin: Mittwoch, 27.01., in Praktikumszeit (*Slots werden noch bekannt gegeben*)
 - Video: 10 Minuten Dauer (pro Team)
 - Anschließend kurzes Q&A (Fragen zum Snake-Mini-Projekt, pro Team)

- Vorführung des Videos und die Q&A findet pro Team statt, Anwesenheit erforderlich
- Abgabe des Videos zum Snake-Mini-Projekt bis Montag, 25.01., 09:00 Uhr im [ILIAS](#).

1.6.2. Gesamtnote: Parcoursprüfung

Sie können die Prüfung in der ersten oder in der zweiten Prüfungsphase ablegen. Die Stationen der Parcoursprüfung sind je nach Prüfungsphase unterschiedlich gestaltet und in sich geschlossen (kein Übertrag):

- **Prüfungsphase I:** Vier Stationen (digitale E-Assessments im B40 mit je 30 Minuten Dauer), beste drei Ergebnisse ergeben die Note
 - Station 1: Mittwoch, 11.11., in Praktikumszeit (*Slots werden noch bekannt gegeben*)
 - Station 2: Mittwoch, 02.12., in Praktikumszeit (*Slots werden noch bekannt gegeben*)
 - Station 3: Mittwoch, 06.01., in Praktikumszeit (*Slots werden noch bekannt gegeben*)
 - Station 4: Im ersten Prüfungszeitraum, Termin wird vom Prüfungsamt bekanntgegeben
- **Prüfungsphase II:** Digitale Klausur im B40, Dauer 120 Minuten, Termin wird vom Prüfungsamt bekanntgegeben

1.6.3. Hinweise

- Die Bearbeitung der Aufgaben erfolgt im Team
- Ein Team umfasst 3 Personen
- Die Post Mortems sind individuell zu erstellen und abzugeben
- Das Video zum Projekt ist pro Team zu erstellen und einmal abzugeben unter Angabe der Teammitglieder
- “Aktive Beteiligung” umfasst Anwesenheit und sachbezogene Beiträge; Anwesenheit/Beteiligung werden dokumentiert
- **Post Mortem:** Jede Person beschreibt individuell(!) die Teilnahme an den Edmonton/Minden-Meetings zurückblickend mit mind. 150 bis max. 400 Wörtern (Nutzlast! Überschriften und Links zählen nicht mit). Gehen Sie dabei aussagekräftig und nachvollziehbar auf folgende Punkte ein:
 1. **Zusammenfassung:** Was wurde auf dem Meeting besprochen?
 2. **Details:** Kurze Beschreibung besonders interessanter Aspekte.
 3. **Reflexion:** Was war der schwierigste Teil? Wie haben Sie dieses Problem gelöst?
 4. **Reflexion:** Was haben Sie gelernt oder (besser) verstanden?

Die Post Mortems geben Sie bitte pro Person bis spätestens zur jeweiligen Deadline im [ILIAS](#) ab.

1.7. Materialien

1. “[Crafting Interpreters](#)”. Nystrom, R., Genever Benning, 2021. ISBN [978-0-9905829-3-9](#). [Online](#).
2. “[Introduction to Compiler Design](#)”. Mogensen, T.A., Springer International, 2024. ISBN [978-3-031-46460-7](#).

1.8. Förderungen und Kooperationen

1.8.1. Kooperation mit University of Alberta, Edmonton (Kanada)

Über das Projekt “[We CAN virtuOWL](#)” der Fachhochschule Bielefeld ist im Frühjahr 2021 eine Kooperation mit der [University of Alberta](#) (Edmonton/Alberta, Kanada) im Modul “Compilerbau” gestartet.

Wir freuen uns, auch in diesem Semester wieder drei gemeinsame Sitzungen für beide Hochschulen anbieten zu können. (Diese Termine werden in englischer Sprache durchgeführt.)

1.9. LICENSE



Unless otherwise noted, [this work](#) by [BC George](#), [Carsten Gips](#) and [contributors](#) is licensed under [CC BY-SA 4.0](#). See the [credits](#) for a detailed list of contributing projects.

Teil II.

Vorlesungsunterlagen

2. Überblick

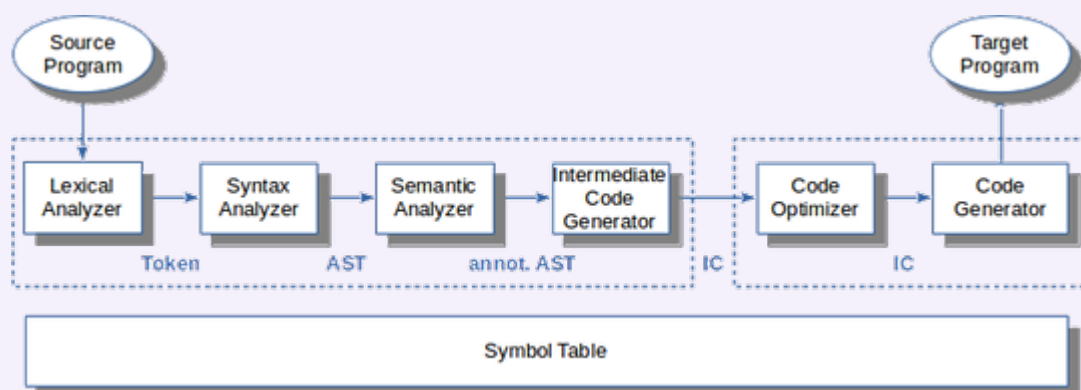
Was ist ein Compiler? Welche Bausteine lassen sich identifizieren, welche Aufgaben haben diese?

2.1. Struktur eines Compilers

TL;DR

Compiler übersetzen (formalen) Text in ein anderes Format.

Typischerweise kann man diesen Prozess in verschiedene Stufen/Phasen einteilen. Dabei verarbeitet jede Phase den Output der vorangegangenen Phase und erzeugt ein (kompakteres) Ergebnis, welches an die nächste Phase weitergereicht wird. Dabei nimmt die Abstraktion von Stufe zu Stufe zu: Der ursprüngliche Input ist ein Strom von Zeichen, daraus wird ein Strom von Wörtern (Token), daraus ein Baum (Parse Tree), Zwischencode (IC), ...



Die gezeigten Phasen werden traditionell unterschieden. Je nach Aufgabe können verschiedene Stufen zusammengefasst werden oder sogar gar nicht auftreten.

Videos

- [VL Überblick](#)

2.1.1. Sprachen verstehen, Texte transformieren

The cat runs quickly.

=> Struktur? Bedeutung?

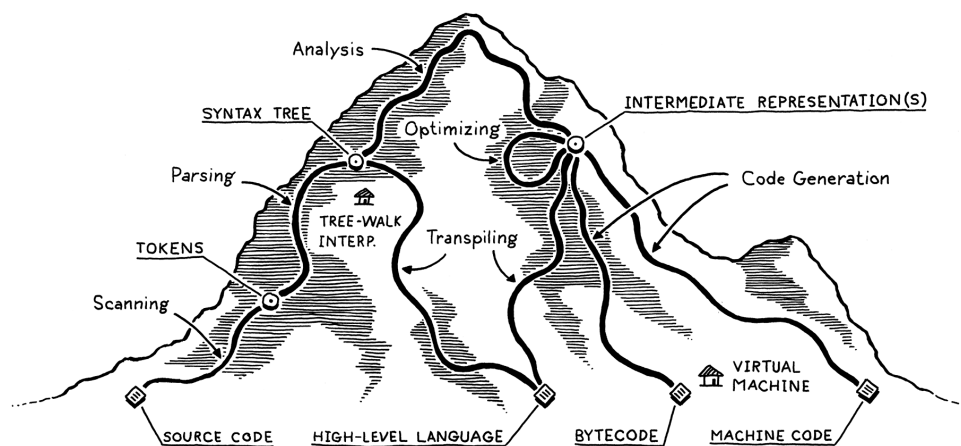
Wir können hier (mit steigender Abstraktionsstufe) unterscheiden:

- Sequenz von Zeichen
- Wörter: Zeichenketten mit bestimmten Buchstaben, getrennt durch bestimmte andere Zeichen; Wörter könnten im Wörterbuch nachgeschlagen werden
- Sätze: Anordnung von Wörtern nach einer bestimmten Grammatik, Grenze: Satzzeichen

Hier (vereinfacht): Ein Satz besteht aus Subjekt und Prädikat. Das Subjekt besteht aus einem oder keinen Artikel und einem Substantiv. Das Prädikat besteht aus einem Verb und einem oder keinem Adverb.

- Sprache: Die Menge der in einer Grammatik erlaubten Sätze

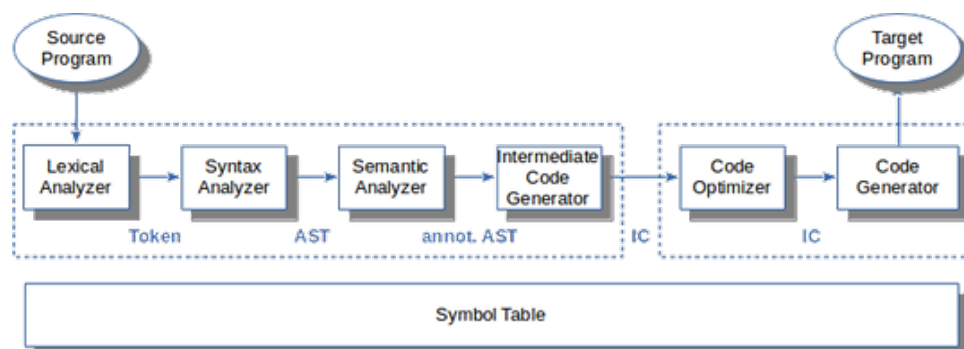
2.1.2. Compiler: Big Picture



Quelle: A Map of the Territory (mountain.png) by Bob Nystrom on Github.com (MIT)

Begriffe und Phasen

Die obige Bergsteige-Metapher kann man in ein nüchternes Ablaufdiagramm mit verschiedenen Stufen und den zwischen den Stufen ausgetauschten Artefakten übersetzen:



2.1.2.1. Frontend, Analyse

Die ersten Stufen eines Compilers, die mit der **Analyse** des Inputs beschäftigt sind. Dies sind in der Regel der Scanner, der Parser und die semantische Analyse.

- Scanner, Lexer, Tokenizer, Lexikalische Analyse

Zerteilt den Zeichenstrom in eine Folge von Wörtern. Mit regulären Ausdrücken kann definiert werden, was Klassen gültiger Wörter ("Token") sind. Ein Token hat i.d.R. einen Namen und einen Wert.

- Parser, Syntaxanalyse

Der Parser erhält als Eingabe die Folge der Token und versucht mit Hilfe einer Grammatik zu bestimmen, ob es sich bei der Tokensequenz um gültige Sätze im Sinne der Grammatik handelt. Hier gibt es viele Algorithmen, die im Wesentlichen in die Klassen "top-down" und "bottom-up" fallen.

- Semantische Analyse, Kontexthandling

In den vorigen Stufen wurde eher lokal gearbeitet. Hier wird über den gesamten Baum und die Symboltabelle hinweg geprüft, ob beispielsweise Typen korrekt verwendet wurden, in welchen Scope ein Name gehört etc. Mit diesen Informationen wird der AST angereichert.

- Symboltabellen

Datenstrukturen, um Namen, Werte, Scopes und weitere Informationen zu speichern. Die Symboltabellen werden vor allem beim Parsen befüllt und bei der semantischen Analyse gelesen, aber auch der Lexer benötigt u.U. diese Informationen.

2.1.2.2. Backend, Synthese

Die hinteren Stufen eines Compilers, die mit der **Synthese** der Ausgabe beschäftigt sind. Dies sind in der Regel verschiedene Optimierungen und letztlich die Code-Generierung

- Codegenerierung

Erzeugung des Zielprogramms aus der (optimierten) Zwischendarstellung. Dies ist oft Maschinencode, kann aber auch C-Code oder eine andere Ziel-Sprache sein.

- Optimierung

Diverse Maßnahmen, um den resultierenden Code kleiner und/oder schneller zu gestalten.

- Symboltabellen

Datenstrukturen, um Namen, Werte, Scopes und weitere Informationen zu speichern. Die Symboltabellen werden vor allem beim Parsen befüllt und bei der semantischen Analyse gelesen, aber auch der Lexer benötigt u.U. diese Informationen.

2.1.2.3. Weitere Begriffe

- Parse Tree, Concrete Syntax Tree

Repräsentiert die Struktur eines Satzes, wobei jeder Knoten dem Namen einer Regel der Grammatik entspricht. Die Blätter bestehen aus den Token samt ihren Werten.

- AST, (Abstract) Syntax Tree

Vereinfachte Form des Parse Tree, wobei der Bezug auf die Element der Grammatik (mehr oder weniger) weggelassen wird.

- Annotierter AST

Anmerkungen am AST, die für spätere Verarbeitungsstufen interessant sein könnten: Typ-Informationen, Optimierungsinformationen, ...

- Zwischen-Code, IC

Zwischensprache, die abstrakter ist als die dem AST zugrunde liegenden Konstrukte der Ausgangssprache. Beispielsweise könnten **while**-Schleifen durch entsprechende Label und Sprünge ersetzt werden. Wie genau dieser Zwischen-Code aussieht, muss der Compilerdesigner entscheiden. Oft findet man den Assembler-ähnlichen "3-Adressen-Code".

- Sprache

Eine Sprache ist eine Menge gültiger Sätze. Die Sätze werden aus Wörtern gebildet, diese wiederum aus Zeichenfolgen.

- Grammatik

Eine Grammatik beschreibt formal die Syntaxregeln für eine Sprache. Jede Regel in der Grammatik beschreibt dabei die Struktur eines Satzes oder einer Phrase.

2.1.3. Lexikalische Analyse: Wörter ("Token") erkennen

Die lexikalische Analyse (auch *Scanner* oder *Lexer* oder *Tokenizer* genannt) zerteilt den Zeichenstrom in eine Folge von Wörtern ("Token"). Die geschieht i.d.R. mit Hilfe von *regulären Ausdrücken*.

Dabei müssen unsinnige/nicht erlaubte Wörter erkannt werden.

Überflüssige Zeichen (etwa Leerzeichen) werden i.d.R. entfernt.

```
1  sp = 100;
```

```
1  <ID, sp>, <OP, =>, <INT, 100>, <SEM>
```

Anmerkung: In der obigen Darstellung werden die Werte der Token ("Lexeme") zusammen mit den Token "gespeichert". Alternativ können die Werte der Token auch direkt in der Symboltabelle gespeichert werden und in den Token nur der Verweis auf den jeweiligen Eintrag in der Tabelle.

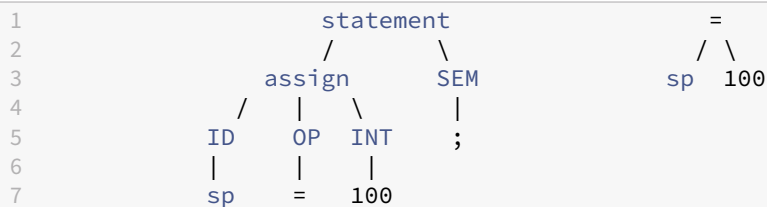
2.1.4. Syntaxanalyse: Sätze erkennen

In der Syntaxanalyse (auch *Parser* genannt) wird die Tokensequenz in gültige Sätze unterteilt. Dazu werden in der Regel *kontextfreie Grammatiken* und unterschiedliche Parsing-Methoden (*top-down*, *bottom-up*) genutzt.

Dabei müssen nicht erlaubte Sätze erkannt werden.

```
1 <ID, sp>, <OP, =>, <INT, 100>, <SEM>
```

```
1 statement : assign SEM ;
2 assign : ID OP INT ;
```



Mit Hilfe der Produktionsregeln der Grammatik wird versucht, die Tokensequenz zu erzeugen. Wenn dies gelingt, ist der Satz (also die Tokensequenz) ein gültiger Satz im Sinne der Grammatik. Dabei sind die Token aus der lexikalischen Analyse die hier betrachteten Wörter!

Dabei entsteht ein sogenannter *Parse-Tree* (oder auch “*Syntax Tree*”; in der obigen Darstellung der linke Baum). In diesen Bäumen spiegeln sich die Regeln der Grammatik wider, d.h. zu einem Satz kann es durchaus verschiedene Parse-Trees geben.

Beim AST (“*Abstract Syntax Tree*”) werden die Knoten um alle später nicht mehr benötigten Informationen bereinigt (in der obigen Darstellung der rechte Baum).

Anmerkung: Die Begriffe werden oft nicht eindeutig verwendet. Je nach Anwendung ist das Ergebnis des Parsers ein AST oder ein Parse-Tree.

Anmerkung: Man könnte statt **OP** auch etwa ein **ASSIGN** nutzen und müsste dann das “=” nicht extra als Inhalt speichern, d.h. man würde die Information im Token-Typ kodieren.

2.1.5. Vorschau: Parser implementieren

```
1 stat : assign | ifstat | ... ;
2 assign : ID '=' expr ';' ;
```

```

1 void stat() {
2     switch (<<current token>>) {
3         case ID : assign(); break;
4         case IF : ifstat(); break;
5         ...
6         default : <<raise exception>>
7     }
8 }
9 void assign() {
10    match(ID);
11    match('=');
12    expr();
13    match(';');
14 }
```

Der gezeigte Parser ist ein sogenannter “LL(1)”-Parser und geht von oben nach unten vor, d.h. ist ein Top-Down-Parser.

Nach dem Betrachten des aktuellen Tokens wird entschieden, welche Alternative vorliegt und in die jeweilige Methode gesprungen.

Die `match()`-Methode entspricht dabei dem Erzeugen von Blättern, d.h. hier werden letztlich die Token der Grammatik erkannt.

2.1.6. Semantische Analyse: Bedeutung erkennen

In der semantischen Analyse (auch *Context Handling* genannt) wird der AST zusammen mit der Symboltabelle geprüft. Dabei spielen Probleme wie Scopes, Namen und Typen eine wichtige Rolle.

Die semantische Analyse ist direkt vom Programmierparadigma der zu übersetzenden Sprache abhängig, d.h. müssen wir beispielsweise das Konzept von Klassen verstehen?

Als Ergebnis dieser Phase entsteht typischerweise ein *annotierter AST*.

```

1 {
2   int x = 42;
3   {
4     int x = 7;
5     x += 3;    // ???
6   }
7 }
```

```

1
2  sp = 100;
3
4
5
6
```

```

graph TD
    A["= {type: real, loc: tmp1}"] --> B["sp {type: real, loc: var b}"]
    A --> C["inttofloat"]
    C --> D["100"]
```

2.1.7. Zwischencode generieren

Aus dem annotierten AST wird in der Regel ein Zwischencode (“*Intermediate Code*”, auch “IC”) generiert. oft findet man hier den Assembler-ähnlichen “3-Adressen-Code”, in manchen Compilern wird als IC aber auch der AST selbst genutzt.

```

1
2
3
4
5
6
```

```

graph TD
    A["= {type: real, loc: tmp1}"] --> B["sp {type: real, loc: var b}"]
    A --> C["inttofloat"]
    C --> D["100"]
```

=> `t1 = inttofloat(100)`

2.1.8. Code optimieren

An dieser Stelle verlassen wir das Compiler-Frontend und begeben uns in das sogenannte *Backend*. Die Optimierung des Codes kann sehr unterschiedlich ausfallen, beispielsweise kann man den Zwischencode selbst optimieren, dann nach sogenanntem "Targetcode" übersetzen und diesen weiter optimieren, bevor das Ergebnis im letzten Schritt in Maschinencode übersetzt wird.

Die Optimierungsphase ist sehr stark abhängig von der Zielhardware. Hier kommen fortgeschrittene Mengen- und Graphalgorithmen zur Anwendung. Die Optimierung stellt den wichtigsten Teil aktueller Compiler dar.

Aus zeitlichen und didaktischen Gründen werden wir in dieser Veranstaltung den Fokus auf die Frontend-Phasen legen und die Optimierung nur grob streifen.

```
t1 = inttfloat(100) => t1 = 100.0
```

```
x = y*0; => x = 0;
```

2.1.9. Code generieren

- Maschinencode:

```
1  STD  t1, 100.0
```

- Andere Sprache:

- Bytecode
- C
- ...

2.1.10. Probleme

```
1  5*4+3
```

AST?

Problem: Vorrang von Operatoren

- Variante 1: $+(*(5, 4), 3)$
- Variante 2: $*(5, +(4, 3))$

```
1  stat : expr ';'
2      | ID '(' ')' ';'
3      ;
4  expr : ID '(' ')'
5      | INT
6      ;
```

2.1.11. Wrap-Up

- Compiler übersetzen Text in ein anderes Format
- Typische Phasen:
 1. Lexikalische Analyse
 2. Syntaxanalyse
 3. Semantische Analyse
 4. Generierung von Zwischencode
 5. Optimierung des (Zwischen-) Codes
 6. Codegenerierung

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k2: Ich kann die Struktur eines Compilers und die verschiedenen Phasen und deren Aufgaben erklären

2.2. Bandbreite der Programmiersprachen

TL;DR

Am Beispiel des Abzählreims “99 Bottles of Beer” werden (ganz kurz) verschiedene Programmiersprachen betrachtet. Jede der Sprachen hat ihr eigenes Sprachkonzept (Programmierparadigma) und auch ein eigenes Typ-System sowie ihre eigene Strategie zur Speicherverwaltung und Abarbeitung. Auch wenn die Darstellung längst nicht vollständig ist, macht sie doch deutlich, dass Compiler teilweise sehr unterschiedliche Konzepte “verstehen” müssen.

Videos

- [VL Programmiersprachen](#)

2.2.1. 99 Bottles of Beer

99 bottles of beer on the wall, 99 bottles of beer. Take one down and pass it around, 98 bottles of beer on the wall.

98 bottles of beer on the wall, 98 bottles of beer. Take one down and pass it around, 97 bottles of beer on the wall.

[...]

2 bottles of beer on the wall, 2 bottles of beer. Take one down and pass it around, 1 bottle of beer on the wall.

1 bottle of beer on the wall, 1 bottle of beer. Take one down and pass it around, no more bottles of beer on the wall.

No more bottles of beer on the wall, no more bottles of beer. Go to the store and buy some more, 99 bottles of beer on the wall.

Quelle: Abzählreim "99 Bottles of Beer" nach "Lyrics of the song 99 Bottles of Beer" on 99-bottles-of-beer.net

2.2.2. Imperativ, Hardwarenah: C

```

1  #define MAXBEER (99)
2  void chug(int beers);
3  main() {
4      register beers;
5      for(beers = MAXBEER; beers; chug(beers--)) puts("");
6      puts("\nTime to buy more beer!\n");
7  }
8  void chug(register beers) {
9      char howmany[8], *s;
10     s = beers != 1 ? "s" : "";
11     printf("%d bottle%s of beer on the wall,\n", beers, s);
12     printf("%d bottle%s of beeeer . . . ,\n", beers, s);
13     printf("Take one down, pass it around,\n");
14     if(--beers) sprintf(howmany, "%d", beers); else strcpy(howmany, "No more");
15     s = beers != 1 ? "s" : "";
16     printf("%s bottle%s of beer on the wall.\n", howmany, s);
17 }

```

Quelle: "Language C" by Bill Wein on 99-bottles-of-beer.net

- Imperativ
- Procedural
- Statisches Typsystem
- Ressourcenschonend, aber "unsicher": Programmierer muss wissen, was er tut
- Relativ hardwarenah
- Einsatz: Betriebssysteme, Systemprogrammierung

2.2.3. Imperativ, Objektorientiert: Java

```

1  class bottles {
2      public static void main(String args[]) {
3          String s = "s";
4          for (int beers=99; beers>-1;) {
5              System.out.print(beers + " bottle" + s + " of beer on the wall, ");

```

```

6      System.out.println(beers + " bottle" + s + " of beer, ");
7      if (beers==0) {
8          System.out.print("Go to the store, buy some more, ");
9          System.out.println("99 bottles of beer on the wall.\n");
10         System.exit(0);
11     } else
12         System.out.print("Take one down, pass it around, ");
13     s = (--beers == 1)?"":"s";
14     System.out.println(beers + " bottle" + s + " of beer on the wall.\n");
15 }
16 }
17 }

```

Quelle: "Language Java" by Sean Russell on 99-bottles-of-beer.net

- Imperativ
- Objektorientiert
- Multi-Threading
- Basiert auf C/C++
- Statisches Typsystem
- Automatische Garbage Collection
- "Sichere" Architektur: Laufzeitumgebung fängt viele Probleme ab
- Architekturneutral: Nutzt Bytecode und eine JVM
- Einsatz: High-Level All-Purpose Language

2.2.4. Logisch: Prolog

```

1  bottles :-
2      bottles(99).
3
4  bottles(1) :-
5      write('1 bottle of beer on the wall, 1 bottle of beer, '), nl,
6      write('Take one down, and pass it around, '), nl,
7      write('Now they are all gone. '), nl,!.
8  bottles(X) :-
9      write(X), write(' bottles of beer on the wall, '), nl,
10     write(X), write(' bottles of beer, '), nl,
11     write('Take one down and pass it around, '), nl,
12     NX is X - 1,
13     write(NX), write(' bottles of beer on the wall. '), nl, nl,
14     bottles(NX).

```

Quelle: "Language Prolog" by M@ on 99-bottles-of-beer.net

- Deklarativ
- Logisch: Definition von Fakten und Regeln; eingebautes Beweissystem

- Einsatz: Theorem-Beweisen, Natural Language Programming (NLP), Expertensysteme, ...

2.2.5. Funktional: Haskell

```
1 bottles 0 = "no more bottles"
2 bottles 1 = "1 bottle"
3 bottles n = show n ++ " bottles"
4
5 verse 0   = "No more bottles of beer on the wall, no more bottles of beer.\n"
6           ++ "Go to the store and buy some more, 99 bottles of beer on the wall."
7           ++ ""
8 verse n   = bottles n ++ " of beer on the wall, " ++ bottles n ++ " of beer.\n"
9           ++ "Take one down and pass it around, " ++ bottles (n-1)
10                                     ++ " of beer on the wall.\n"
11
12 main     = mapM (putStrLn . verse) [99,98..0]
```

Quelle: "Language Haskell" by Iavor on 99-bottles-of-beer.net

- Deklarativ
- Funktional
- Lazy, pure
- Statisches Typsystem
- Typinferenz
- Algebraische Datentypen, Patternmatching
- Einsatz: Compiler, DSL, Forschung

2.2.6. Brainfuck

[illegible]

Quelle: Screenshot of "Language Brainfuck" by Michal Wojciech Tarnowski on 99-bottles-of-beer.net

- Imperativ
- Feldbasiert (analog zum Band der Turingmaschine)
- 8 Befehle: Zeiger und Zellen inkrementieren/dekrementieren, Aus- und Eingabe, Sprungbefehle

2.2.7. Programmiersprache Lox

```

1 fun fib(x) {
2     if (x == 0) {
3         return 0;
4     } else {
5         if (x == 1) {
6             return 1;
7         } else {
8             fib(x - 1) + fib(x - 2);
9         }
10    }
11 }
12
13 var wuppie = fib;
14 wuppie(4);

```

- Die Sprache “Lox” finden Sie hier: craftinginterpreters.com/the-lox-language.html
- C-ähnliche Syntax
- Imperativ, objektorientiert, Funktionen als *First Class Citizens*, Closures

- Dynamisch typisiert
- Garbage Collector
- Statements und Expressions
- (Kleine) Standardbibliothek eingebaut

Die Sprache ähnelt stark anderen modernen Sprachen und ist gut geeignet, um an ihrem Beispiel Themen wie Scanner/Parser/AST, Interpreter, Object Code und VM zu studieren :)

2.2.8. Wrap-Up

- Compiler übersetzen formalen Text in ein anderes Format
- Berücksichtigung von unterschiedlichen
 - Sprachkonzepten (Programmierparadigmen)
 - Typ-Systemen
 - Speicherverwaltungsstrategien
 - Abarbeitungsstrategien

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k1: Ich kenne verschiedene Beispiele für verschiedene Programmiersprachen und Paradigmen

2.3. Anwendungen

TL;DR

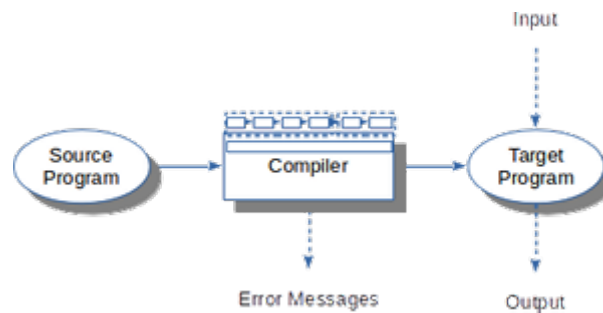
Es gibt verschiedene Anwendungsmöglichkeiten für Compiler. Je nach Bedarf wird dabei die komplette Toolchain durchlaufen oder es werden Stufen ausgelassen. Häufig genutzte Varianten sind dabei:

- “Echte” Compiler: Übersetzen Sourcecode nach ausführbarem Maschinencode
- Interpreter: Interaktive Ausführung von Sourcecode
- Virtuelle Maschinen als Zwischending zwischen Compiler und Interpreter
- Transpiler: Übersetzen formalen Text nach formalem Text
- Analysetools: Parsen den Sourcecode, werten die Strukturen aus

Videos

- [VL Anwendungen](#)

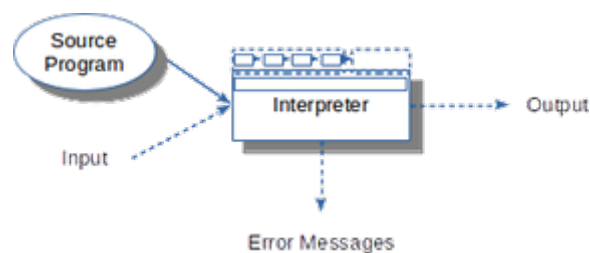
2.3.1. Anwendung: Compiler



Wie oben diskutiert: Der Sourcecode durchläuft alle Phasen des Compilers, am Ende fällt ein ausführbares Programm heraus. Dieses kann man starten und ggf. mit Inputdaten versehen und erhält den entsprechenden Output. Das erzeugte Programm läuft i.d.R. nur auf einer bestimmten Plattform.

Beispiele: gcc, clang, ...

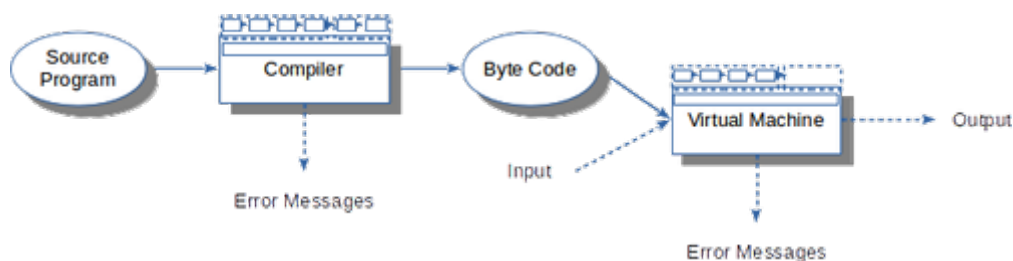
2.3.2. Anwendung: Interpreter



Beim Interpreter durchläuft der Sourcecode nur das Frontend, also die Analyse. Es wird kein Code erzeugt, stattdessen führt der Interpreter die Anweisungen im AST bzw. IC aus. Dazu muss der Interpreter mit den Eingabedaten beschickt werden. Typischerweise hat man hier eine "Read-Eval-Print-Loop" (REPL).

Beispiele: Python

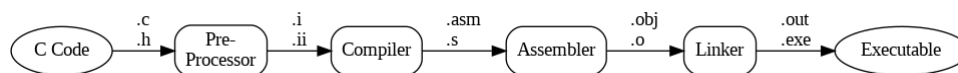
2.3.3. Anwendung: Virtuelle Maschinen



Hier liegt eine Art Mischform aus Compiler und Interpreter vor: Der Compiler übersetzt den Quellcode in ein maschinenunabhängiges Zwischenformat ("Byte-Code"). Dieser wird von der virtuellen Maschine ("VM") gelesen und ausgeführt. Die VM kann also als Interpreter für Byte-Code betrachtet werden.

Beispiel: Java mit seiner JVM

2.3.4. Anwendung: C-Toolchain

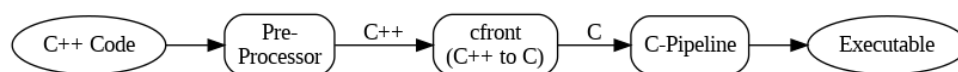


Erinnern Sie sich an die LV "Systemprogrammierung" im dritten Semester :-)

Auch wenn es so aussieht, als würde der C-Compiler aus dem Quelltext direkt das ausführbare Programm erzeugen, finden hier dennoch verschiedene Stufen statt. Zuerst läuft ein Präprozessor über den Quelltext und ersetzt alle `#include` und `#define` etc., danach arbeitet der C-Compiler, dessen Ausgabe wiederum durch einen Assembler zu ausführbarem Maschinencode transformiert wird.

Beispiele: gcc, clang, ...

2.3.5. Anwendung: C++-Compiler



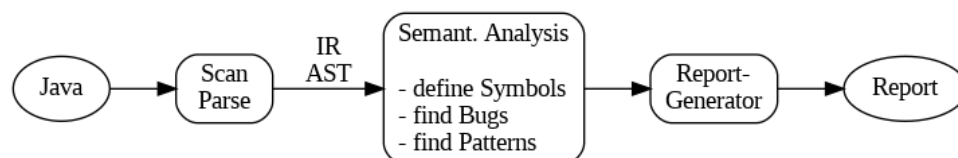
C++ hat meist keinen eigenen (vollständigen) Compiler :-)

In der Regel werden die C++-Konstrukte durch `cfront` nach C übersetzt, so dass man anschließend auf die etablierten Tools zurückgreifen kann.

Dieses Vorgehen werden Sie relativ häufig finden. Vielleicht sogar in Ihrem Projekt ...

Beispiel: g++

2.3.6. Anwendung: Bugfinder



Tools wie FindBugs analysieren den (Java-) Quellcode und suchen nach bekannten Fehlermustern. Dazu benötigen sie nur den Analyse-Teil eines Compilers!

Auf dem AST kann dann nach vorab definierten Fehlermustern gesucht werden (Stichwort “Graphmatching”). Dazu fällt die semantische Analyse entsprechend umfangreicher aus als normal.

Zusätzlich wird noch eine Reporting-Komponente benötigt, da die normalen durch die Analyseketten erzeugten Fehlermeldungen nicht helfen (bzw. sofern der Quellcode wohlgeformter Code ist, würden ja keine Fehlermeldungen durch die Analyseeinheit generiert).

Beispiele: SpotBugs, Checkstyle, ESLint, ...

2.3.7. Anwendung: Pandoc

Pandoc ist ein universeller und modular aufgebauter Textkonverter, der mit Hilfe verschiedener *Reader* unterschiedliche Textformate einlesen und in ein Zwischenformat (hier JSON) transformieren kann. Über verschiedene *Writer* können aus dem Zwischenformat dann Dokumente in den gewünschten Zielformaten erzeugt werden.

Die Reader entsprechen der Analyse-Phase und die Writer der Synthese-Phase eines Compilers. Anstelle eines ausführbaren Programms (Maschinencode) wird ein anderes Textformat erstellt/ausgegeben.

Beispielsweise wird aus diesem Markdown-Schnipsel ...

```
1 Dies ist ein Satz mit
2 * einem Stichpunkt, und
3 * einem zweiten Stichpunkt.
```

... dieses Zwischenformat erzeugt, ...

```
1 {"blocks":[{"t":"Para","c":[{"t":"Str","c":"Dies"}, {"t":"Space"},
2 {"t":"Str","c":"ist"}, {"t":"Space"}, {"t":"Str","c":"ein"},
3 {"t":"Space"}, {"t":"Str","c":"Satz"}, {"t":"Space"},
4 {"t":"Str","c":"mit"}]},
5 {"t":"BulletList","c":[{"t":"Plain","c":[{"t":"Str","c":"einem"}, {"t":"Space"}, {"t":"Str","c":"Stichpunkt,"}, {"t":"Space"}, {"t":"Str","c":"und"}]}, [{"t":"Plain","c":[{"t":"Str","c":"einem"}, {"t":"Space"}, {"t":"Str","c":"zweiten"}, {"t":"Space"}, {"t":"Str","c":"Stichpunkt."}]}]}], "pandoc-api-version": [1, 17, 0, 4], "meta": {}}
```

... und daraus schließlich dieser TeX-Code.

```
1 Dies ist ein Satz mit
2 \begin{itemize}
3 \tightlist
4 \item einem Stichpunkt, und
5 \item einem zweiten Stichpunkt.
6 \end{itemize}
```

Im Prinzip ist Pandoc damit ein Beispiel für Compiler, die aus einem formalen Text nicht ein ausführbares Programm erzeugen (Maschinencode), sondern einen anderen formalen Text. Dieser werden häufig auch “Transpiler” genannt.

Weitere Beispiele:

- Lexer-/Parser-Generatoren: ANTLR, Flex, Bison, ...: formale Grammatik nach Sourcecode

- CoffeeScript: CoffeeScript (eine Art “JavaScript light”) nach JavaScript
- Emscripten: C/C++ nach LLVM nach WebAssembly (tatsächlich kann LLVM-IR auch direkt als Input verwendet werden)
- Fitness: Word/Wiki nach ausführbare Unit-Tests

2.3.8. Was bringt mir das?

Beschäftigung mit dem schönsten Thema in der Informatik ;-)

2.3.8.1. Auswahl einiger Gründe für den Besuch des Moduls “Compilerbau”

- Erstellung eigener kleiner Interpreter/Compiler
 - Einlesen von komplexen Daten
 - DSL als Brücke zwischen Stakeholdern
 - DSL zum schnelleren Programmieren (denken Sie etwa an [CoffeeScript](#) ...)
- Wie funktionieren FindBugs, Lint und ähnliche Tools?
 - Statische Codeanalyse: Dead code elimination
- Language-theoretic Security: [LangSec](#)
- Verständnis für bestimmte Sprachkonstrukte und -konzepte (etwa [virtual](#) in C++)
- Vertiefung durch Besuch “echter” Compilerbau-Veranstaltungen an Uni möglich :-)
- Wie funktioniert:
 - ein Python-Interpreter?
 - das Syntaxhighlighting in einem Editor oder in Doxygen?
 - ein Hardwarecompiler (etwa VHDL)?
 - ein Text-Formatierer (TeX, LaTeX, ...)?
 - CoffeeScript oder Emscripten?
- Wie kann man einen eigenen Compiler/Interpreter basteln, etwa für
 - MiniJava (mit C-Backend)
 - Brainfuck
 - Übersetzung von JSON nach XML
- Um ein tiefes Verständnis von Programmiersprachen zu erlangen, ist eine Beschäftigung mit ihrer Implementierung unerlässlich.
- Viele Grundtechniken der Informatik und elementare Datenstrukturen wie Keller, Listen, Abbildungen, Bäume, Graphen, Automaten etc. finden im Compilerbau Anwendung. Dadurch schließt sich in gewisser Weise der Kreis in der Informatikausbildung ...
- Aufgrund seiner Reife gibt es hervorragende Beispiele von formaler Spezifikation im Compilerbau.
- Mit dem Gebiet der formalen Sprachen berührt der Compilerbau interessante Aspekte moderner Linguistik. Damit ergibt sich letztlich eine Verbindung zur KI ...
- Die Unterscheidung von Syntax und Semantik ist eine grundlegende Technik in fast allen formalen Systemen.

2.3.8.2. Parser-Generatoren (Auswahl)

Diese Tools könnte man beispielsweise nutzen, um seine eigene Sprache zu basteln.

- ANTLR (ANother Tool for Language Recognition) is a powerful parser generator for reading, processing, executing, or translating structured text or binary files: github.com/antlr/antlr4
- Grammars written for ANTLR v4; expectation that the grammars are free of actions: github.com/antlr/grammars-v4
- An incremental parsing system for programmings tools: github.com/tree-sitter/tree-sitter
- Flex, the Fast Lexical Analyzer - scanner generator for lexing in C and C++: github.com/westes/flex
- Bison is a general-purpose parser generator that converts an annotated context-free grammar into a deterministic LR or generalized LR (GLR) parser employing LALR(1) parser tables: gnu.org/software/bison
- Parser combinators for binary formats, in C: github.com/UpstandingHackers/hammer
- Eclipse Xtext is a language development framework: github.com/eclipse/xtext

2.3.8.3. Statische Analyse, Type-Checking und Linter

Als Startpunkt für eigene Ideen. Oder Verbessern/Erweitern der Projekte ...

- Pluggable type-checking for Java: github.com/typetools/checker-framework
- SpotBugs is FindBugs' successor. A tool for static analysis to look for bugs in Java code: github.com/spotbugs/spotbugs
- An extensible cross-language static code analyzer: github.com/pmd/pmd
- Checkstyle is a development tool to help programmers write Java code that adheres to a coding standard: github.com/checkstyle/checkstyle
- JaCoCo - Java Code Coverage Library: github.com/jacoco/jacoco
- Sanitizers: memory error detector: github.com/google/sanitizers
- JSHint is a tool that helps to detect errors and potential problems in your JavaScript code: github.com/jshint/jshint
- Haskell source code suggestions: github.com/ndmitchell/hlint
- Syntax checking hacks for vim: github.com/vim-syntastic/syntastic

2.3.8.4. DSL (Domain Specific Language)

- NVIDIA Material Definition Language SDK: github.com/NVIDIA/MDL-SDK
- FitNesse – The Acceptance Test Wiki: github.com/unclebob/fitnesse

Hier noch ein Framework, welches auf das Erstellen von DSL spezialisiert ist:

- Eclipse Xtext is a language development framework: github.com/eclipse/xtext

2.3.8.5. Konverter von X nach Y

- Emscripten: An LLVM-to-JavaScript Compiler: github.com/kripken/emscripten
- “Unfancy JavaScript”: github.com/jashkenas/coffeescript

- Universal markup converter: github.com/jgm/pandoc
- Übersetzung von JSON nach XML

2.3.8.6. Odds and Ends

- How to write your own compiler: staff.polito.it/silvano.rivoira/HowToWriteYourOwnCompiler.htm
- Building a modern functional compiler from first principles: github.com/sdiehl/write-you-a-haskell
- Language-theoretic Security: [LangSec](#)
- Generierung von automatisierten Tests mit [Esprima](#): heise.de/-4129726
- Eigener kleiner Compiler/Interpreter, etwa für
 - MiniJava mit C-Backend oder sogar [LLVM](#)-Backend
 - Brainfuck

2.3.8.7. Als weitere Anregung: Themen der Mini-Projekte im W17

- Java2UMLet
- JavaDoc-to-Markdown
- Validierung und Übersetzung von Google Protocol Buffers v3 nach JSON
- svg2tikz
- SwaggerLang – Schreiben wie im Tagebuch
- Markdown zu LaTeX
- JavaDocToLaTeX
- MySQL2REDIS-Parser

2.3.9. Wrap-Up

- Compiler übersetzen formalen Text in ein anderes Format
- Nicht alle Stufen kommen immer vor => unterschiedliche Anwendungen
 - “Echte” Compiler: Sourcecode nach Maschinencode
 - Interpreter: Interaktive Ausführung
 - Virtuelle Maschinen als Zwischending zwischen Compiler und Interpreter
 - Transpiler: formaler Text nach formalem Text
 - Analysetools: Parsen den Sourcecode, werten die Strukturen aus

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k1: Ich kenne verschiedene Anwendungen für Compiler durch Einsatz bestimmter Stufen der Compiler-Pipeline

3. Reguläre Sprachen, kontextfreie Grammatiken und Sprachen, lexikalische und syntaktische Analyse

In der lexikalischen Analyse soll ein Lexer (auch “Scanner”) den Zeichenstrom in eine Folge von Token zerlegen. Zur Spezifikation der Token werden in der Regel reguläre Ausdrücke verwendet.

In der syntaktischen Analyse arbeitet ein Parser mit dem Tokenstrom, der vom Lexer kommt. Mit Hilfe einer Grammatik wird geprüft, ob gültige Sätze im Sinne der Sprache/Grammatik gebildet wurden. Der Parser erzeugt dabei den Parse-Tree. Man kann verschiedene Parser unterscheiden, beispielsweise die LL- und die LR-Parser.

3.1. Reguläre Sprachen, Ausdrucksstärke (Teil 1)

Weitere Unterlagen

- [Annotierte Folien: Reguläre Sprachen, Ausdrucksstärke](#)

3.1.1. Motivation

3.1.1.1. Was muss ein Compiler wohl als erstes tun?

Hier entsteht ein Tafelbild.

3.1.1.2. Themen für heute

- Lexer
- Endliche Automaten
- Reguläre Sprachen

3.1.2. Endliche Automaten

3.1.2.1. Alphabete

Def.: Ein **Alphabet** Σ ist eine endliche, nicht-leere Menge von Symbolen. Die Symbole eines Alphabets heißen *Buchstaben*.

Def.: Ein **Wort** w über einem Alphabet Σ ist eine endliche Folge von Symbolen aus Σ . ϵ ist das leere Wort. Die Länge $|w|$ eines Wortes w ist die Anzahl von Buchstaben, die es enthält (Kardinalität).

Def.: Eine **Sprache** L über einem Alphabet Σ ist eine Menge von Wörtern über diesem Alphabet. Sprachen können endlich oder unendlich viele Wörter enthalten.

3.1.2.2. State machine

Hier entsteht ein Tafelbild.

3.1.2.3. Deterministische endliche Automaten

Bestimmte State machines:

- Eingaben bestimmen Zustandsübergänge
- Zustandsübergänge sind eindeutig
- Es gibt Anfang(zustand) und End(zustand)e

3.1.2.4. Wie definieren wir das formal?

Hier entsteht ein Tafelbild.

3.1.2.5. Def.: Deterministischer endlicher Automat

Def.: Ein **deterministischer endlicher Automat** (DFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \rightarrow Q$, δ kann partiell sein
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

3.1.2.6. Beispiel

Hier entsteht ein Tafelbild.

3.1.2.7. Eingabewörter statt Buchstaben

Def.: Wir definieren $\delta^* : (Q \times \Sigma^*) \rightarrow Q$: induktiv wie folgt:

- Basis: $\delta^*(q, \epsilon) = q \ \forall q \in Q$
- Induktion: $\delta^*(q, a_1, \dots, a_n) = \delta(\delta^*(q, a_1, \dots, a_{n-1}), a_n)$

Def.: Ein DFA akzeptiert ein Wort $w \in \Sigma^*$ genau dann, wenn $\delta^*(q_0, w) \in F$.

3.1.2.8. Beispiel

Hier entsteht ein Tafelbild.

3.1.2.9. Nichtdeterministische endliche Automaten

Hier entsteht ein Tafelbild.

3.1.2.10. Def.: Nichtdeterministischer Automat

Def.: Ein **nichtdeterministischer endlicher Automat** (NFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \rightarrow Q$
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

3.1.2.11. Akzeptierte Sprachen

Def.: Sei A ein DFA oder ein NFA. Dann ist **$L(A)$** die von A akzeptierte Sprache, d. h.

$$L(A) = \{\text{Wörter } w \mid \delta^*(q_0, w) \in F\}$$

3.1.2.12. Wozu NFAs im Compilerbau?

Pattern Matching (Erkennung von Schlüsselwörtern, Bezeichnen, ...) geht mit NFAs.

NFAs sind so nicht zu programmieren, aber:

Satz: Eine Sprache L wird von einem NFA akzeptiert $\Leftrightarrow L$ wird von einem DFA akzeptiert.

D. h. es existieren Algorithmen zur

- Umwandlung von NFAs in DFAs
- Minimierung von DFAs

3.1.3. Reguläre Sprachen

3.1.3.1. Reguläre Ausdrücke definieren Sprachen

Def.: Induktive Definition von **regulären Ausdrücken** (regex) und der von ihnen repräsentierten Sprache **L**:

- Basis:
 - ϵ und $\emptyset$ sind reguläre Ausdrücke mit $L(\epsilon) = \{\epsilon\}$, $L(\emptyset) = \emptyset$
 - Sei a ein Symbol $\Rightarrow a$ ist ein regex mit $L(a) = \{a\}$
- Induktion: Seien E, F reguläre Ausdrücke. Dann gilt:
 - $E + F$ ist ein regex und bezeichnet die Vereinigung $L(E + F) = L(E) \cup L(F)$
 - EF ist ein regex und bezeichnet die Konkatenation $L(EF) = L(E)L(F)$
 - E^* ist ein regex und bezeichnet die Kleene-Hülle $L(E^*) = (L(E))^*$
 - (E) ist ein regex mit $L((E)) = L(E)$

Vorrangregeln der Operatoren für reguläre Ausdrücke: $*$, Konkatenation, $+$

3.1.3.2. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.3. Wichtige Identitäten

Satz: Sei A ein DFA $\Rightarrow \exists$ regex R mit $L(A) = L(R)$.

Satz: Sei E ein regex $\Rightarrow \exists$ DFA A mit $L(E) = L(A)$.

3.1.3.4. Formale Grammatiken

Hier entsteht ein Tafelbild.

3.1.3.5. Formale Definition formaler Grammatiken

Def.: Eine *formale Grammatik* ist ein 4-Tupel $G = (N, T, P, S)$ aus

- N : endliche Menge von **Nichtterminalen**
- T : endliche Menge von **Terminalen**, $N \cap T = \emptyset$
- $S \in N$: **Startsymbol**
- P : endliche Menge von **Produktionen** der Form

$$X \rightarrow Y \text{ mit } X \in (N \cup T)^* N (N \cup T)^*, Y \in (N \cup T)^*$$

3.1.3.6. Ableitungen

Def.: Sei $G = (N, T, P, S)$ eine Grammatik, sei $\alpha A \beta$ eine Zeichenkette über $(N \cup T)^*$ und sei $A \rightarrow \gamma$ eine Produktion von G .

Wir schreiben: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ ($\alpha A \beta$ leitet $\alpha \gamma \beta$ ab).

Def.: Wir definieren die Relation $\Rightarrow^*$ induktiv wie folgt:

- Basis: $\forall \alpha \in (N \cup T)^* \alpha \Rightarrow^* \alpha$ (Jede Zeichenkette leitet sich selbst ab.)
- Induktion: Wenn $\alpha \Rightarrow^* \beta$ und $\beta \Rightarrow \gamma$ dann $\alpha \Rightarrow^* \gamma$

Def.: Sei $G = (N, T, P, S)$ eine formale Grammatik. Dann ist $L(G) = \{\text{Wörter } w \text{ über } T \mid S \Rightarrow^* w\}$ die von G erzeugte Sprache.

3.1.3.7. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.8. Reguläre Grammatiken

Def.: Eine **reguläre (oder type-3-) Grammatik** ist eine formale Grammatik mit den folgenden Einschränkungen:

- Alle Produktionen sind entweder von der Form
 - $X \rightarrow aY$ mit $X \in N, a \in T, Y \in N$ (*rechtsreguläre Grammatik*) oder
 - $X \rightarrow Ya$ mit $X \in N, a \in T, Y \in N$ (*linksreguläre Grammatik*)
- $X \rightarrow \epsilon$ ist erlaubt

3.1.3.9. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.10. Reguläre Sprachen

Satz: Die von endlichen Automaten akzeptierte Sprachklasse, die von regulären Ausdrücken beschriebene Sprachklasse und die von regulären Grammatiken erzeugte Sprachklasse sind identisch und heißen **reguläre Sprachen**.

3.1.4. Wrap-Up

3.1.4.1. Wrap-Up

- Definition und Aufgaben von Lexern
- DFAs und NFAs
- Reguläre Ausdrücke
- Reguläre Grammatiken
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren

Zum Nachlesen

- Aho u. a. (2023): Abschnitt 2.6 und Kapitel 3
- Torczon und Cooper (2012): Kapitel 2
- Parr (2014)

Lernziele

- k1: Ich kenne DFAs
- k1: Ich kenne reguläre Ausdrücke
- k1: Ich kenne reguläre Grammatiken
- k2: Ich kann die Zusammenhänge und Gesetzmäßigkeiten bzgl. der oben genannten Konstrukte an einem Beispiel erklären
- k3: Ich kann für eine Fragestellung DFAs, reguläre Ausdrücke, reguläre Grammatiken entwickeln
- k3: Ich kann einen DFA entwickeln, der alle Schlüsselwörter, Namen und weitere Symbole einer Programmiersprache akzeptiert

3.2. Reguläre Sprachen, Ausdrucksstärke (Teil 2)

Weitere Unterlagen

- [Annotierte Folien: Reguläre Sprachen, Ausdrucksstärke](#)

3.2.1. Wiederholung

3.2.1.1. Endliche Automaten, reguläre Ausdrücke, reguläre Grammatiken, reguläre Sprachen

- Wie sind DFAs und NFAs definiert?
- Was sind reguläre Ausdrücke?
- Was sind formale und reguläre Grammatiken?
- In welchem Zusammenhang stehen all diese Begriffe?

3.2.2. Motivation

3.2.2.1. Was haben reguläre Sprachen mit Compilern zu tun?

Hier entsteht ein Tafelbild.

3.2.2.2. Themen für heute

- Reguläre Sprachen
- Lexer
- Grenzen regulärer Sprachen

3.2.2.3. Wozu reguläre Sprachen im Compilerbau?

Reguläre Ausdrücke

- definieren Schlüsselwörter und alle weiteren Symbole einer Programmiersprache, z. B. den Aufbau von Gleitkommazahlen
- werden (oft von einem Generator) in DFAs umgewandelt
- sind die Basis des *Scanners* oder *Lexers*

3.2.3. Lexer

3.2.3.1. Ein Lexer ist mehr als ein DFA

Ein **Lexer**

- kann aus regulären Ausdrücken automatisch generiert werden
- wandelt mittels DFAs aus regulären Ausdrücken die Folge von Zeichen der Quelldatei in eine Folge von sog. Token um
- bekommt als Input eine Liste von Paaren aus regulären Ausdrücken und Tokennamen, z. B. ("while", WHILE)
- Kommentare und Strings müssen richtig erkannt werden. (Schachtelungen)
- liefert Paare von Token und deren Werte, sofern benötigt, z. B. (WHILE, _), oder (IDENTIFIER, "radius") oder (INTEGERZAHL, "334")

3.2.3.2. Wofür reichen reguläre Sprachen nicht?

Für z. B. alle Sprachen, in deren Wörtern Zeichen über eine Konstante hinaus gezählt werden müssen. Diese Sprachen lassen sich oft mit Variablen im Exponenten beschreiben, die unendlich viele Werte annehmen können.

- $a^i b^{2*i}$ ist nicht regulär

- $a^i b^{2*i}$ für $0 \leq i \leq 3$ ist regulär
- Wo finden sich die oben genannten Variablen bei einem DFA wieder?
- Warum ist die erste Sprache oben nicht regulär, die zweite aber?

3.2.3.3. Wie geht es weiter?

Ein Parser

- führt mit Hilfe des Tokenstreams vom Lexer die Syntaxanalyse durch
- basiert auf einer sog. kontextfreien Grammatik, deren Terminale die Token sind
- liefert die syntaktische Struktur in Form eines Ableitungsbaums (**syntax tree, parse tree**), bzw. einen **AST** (abstract syntax tree) ohne redundante Informationen im Ableitungsbaum (z. B. Semikolons)
- liefert evtl. Fehlermeldungen

3.2.4. Wrap-Up

3.2.4.1. Wrap-Up

- Definition und Aufgaben von Lexern
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren
- Grenzen regulärer Sprachen

Zum Nachlesen

- Aho u. a. (2023): Abschnitt 2.6 und Kapitel 3
- Torczon und Cooper (2012): Kapitel 2
- Parr (2014)

Lernziele

- k1: Ich kenne die Aufgaben eines Lexers
- k1: Ich kenne die Zusammenhänge zwischen DFAs, regulären Ausdrücken, regulären Grammatiken und Lexern
- k2: Ich kann für eine Beispielsprache begründen, warum sie nicht mit einem der oben genannten Mechanismen beschrieben werden kann

3.3. CFG

Weitere Unterlagen

- [Annotierte Folien: CFG, LL-Parser](#)

3.3.1. Wiederholung

3.3.1.1. Endliche Automaten, reguläre Ausdrücke, reguläre Grammatiken, reguläre Sprachen

- Was ist ein Lexer?
- In welchem Zusammenhang stehen Lexer und reguläre Sprachen?
- Was können Lexer nicht?

3.3.2. Motivation

3.3.2.1. Was brauchen wir jetzt?

3.3.2.2. Themen für heute

- PDAs: mächtiger als DFAs, NFAs
- kontextfreie Grammatiken und Sprachen: mächtiger als reguläre Grammatiken und Sprachen
- DPDAs und deterministisch kontextfreie Grammatiken: die Grundlage der Syntaxanalyse im Compilerbau
- Syntaxanalyse

3.3.2.3. Einordnung: Erweiterung der Automatenklasse DFA, um komplexere Sprachen als die regulären akzeptieren zu können

Wir spendieren den DFAs einen möglichst einfachen, aber beliebig großen, Speicher, um zählen und matchen zu können. Wir suchen dabei konzeptionell die "kleinstmögliche" Erweiterung, die die akzeptierte Sprachklasse gegenüber DFAs vergrößert.

- Der konzeptionell einfachste Speicher ist ein Stack. Wir haben keinen wahlfreien Zugriff auf die gespeicherten Werte.
- Es soll eine deterministische und eine indeterministische Variante der neuen Automatenklasse geben.
- In diesem Zusammenhang wird der Stack auch Keller genannt.

3.3.2.4. Kellerautomaten (Push-Down-Automata, PDAs)

Def.: Ein Kellerautomat (PDA) $P = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$ ist ein Septupel mit:

Q :	eine endliche Menge von Zuständen
Σ :	eine endliche Menge von Eingabesymbolen
Γ :	ein endliches Kelleralphabet
δ :	die Übergangsfunktion
	$\delta : Q \times \Sigma \cup \{\epsilon\} \times \Gamma^* \rightarrow 2^{Q \times \Gamma^*}$
q_0 :	der Startzustand
$\perp \in \Gamma$:	der anfängliche Kellerinhalt, symbolisiert den leeren Keller ($\perp = \text{bottom}$)
$F \subseteq Q$:	die Menge von Endzuständen

Abbildung 3.1.: Definition eines PDAs

Ein PDA ist per Definition nichtdeterministisch und kann spontane Zustandsübergänge durchführen.

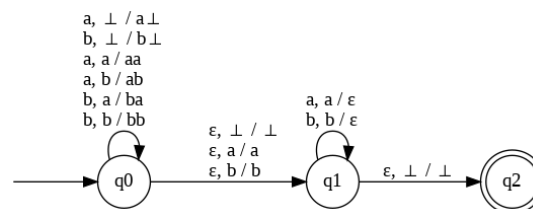
3.3.2.5. Was kann man damit akzeptieren?

Strukturen mit paarweise zu matchenden Symbolen.

Bei jedem Zustandsübergang wird ein Zeichen (oder ϵ) aus der Eingabe gelesen, ein Symbol von Keller genommen. Diese und das Eingabezeichen bestimmen den Folgezustand und eine Zeichenfolge, die auf den Stack gepackt wird. Dabei wird ein Symbol, (z. B. eines, das später mit einem Eingabesymbol zu matchen ist,) auf den Stack gepackt. Soll das automatisch vom Stack genommene Symbol auf dem Stack bleiben, muss es wieder gepusht werden.

3.3.2.6. Beispiel

Ein PDA für $L = \{ww^R \mid w \in \{a, b\}^*\}$:



3.3.2.7. Noch ein Beispiel

Hier entsteht ein Tafelbild.

3.3.2.8. Deterministische PDAs

Def. Ein PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$ ist *deterministisch* : $\Leftrightarrow$

- $\delta(q, a, X)$ hat höchstens ein Element für jedes $q \in Q, a \in \Sigma$ oder ($a = \epsilon$ und $X \in \Gamma$).
- Wenn $\delta(q, a, x)$ nicht leer ist für ein $a \in \Sigma$, dann muss $\delta(q, \epsilon, x)$ leer sein.

Deterministische PDAs werden auch *DPDAs* genannt.

3.3.2.9. Der kleine Unterschied

Satz: Die von DPDAs akzeptierten Sprachen sind eine echte Teilmenge der von PDAs akzeptierten Sprachen.

Die regulären Sprachen sind eine echte Teilmenge der von DPDAs akzeptierten Sprachen.

3.3.3. Kontextfreie Grammatiken und Sprachen

3.3.3.1. Kontextfreie Grammatiken

Def. Eine *kontextfreie* (cf-) Grammatik ist ein 4-Tupel $G = (N, T, P, S)$ mit N, T, S wie in (formalen) Grammatiken und P ist eine endliche Menge von Produktionen der Form:

$X \rightarrow Y$ mit $X \in N, Y \in (N \cup T)^*$.

$\Rightarrow, \Rightarrow^*$ sind definiert wie bei regulären Sprachen. Bei cf-Grammatiken nennt man die Ableitungsbäume oft *Parse trees*.

3.3.3.2. Beispiel

$S \rightarrow a \mid S + S \mid S * S$
Hier entsteht ein Tafelbild.

Ableitungsbäume für $a + a * a$:

3.3.3.3. Was ist hier los?

Hier entsteht ein Tafelbild.

3.3.3.4. Nicht jede kontextfreie Grammatik ist eindeutig

Def.: Gibt es in einer von einer kontextfreien Grammatik erzeugten Sprache ein Wort, für das mehr als ein Ableitungsbaum existiert, so heißt diese Grammatik *mehrdeutig*. Anderenfalls heißt sie *eindeutig*.

Satz: Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.

Satz: Es gibt kontextfreie Sprachen, für die keine eindeutige Grammatik existiert.

3.3.3.5. Kontextfreie Grammatiken und PDAs

Satz: Die kontextfreien Sprachen und die Sprachen, die von PDAs akzeptiert werden, sind dieselbe Sprachklasse.

Satz: Eine von einem DPDA akzeptierte Sprache hat eine eindeutige Grammatik.

Vorgehensweise im Compilerbau: Eine (cf) Grammatik für die gewünschte Sprache definieren und schauen, ob sich daraus ein DPDA generieren lässt (automatisch).

3.3.4. Syntaxanalyse

3.3.4.1. Syntax

Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

3.3.4.2. Ziele der Syntaxanalyse

- Bestimmung der syntaktischen Struktur eines Programms
- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltabelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, ...), aber auch die Gültigkeitsbereiche

3.3.4.3. Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse (= Parser) erstellen lässt
- einen Algorithmus zum Parsen von Programmen mit Hilfe einer solchen Grammatik

3.3.5. Wrap-Up

3.3.5.1. Das sollen Sie mitnehmen

- Die Struktur von gängigen Programmiersprachen lässt sich nicht mit regulären Ausdrücken beschreiben und damit nicht mit DFAs akzeptieren.
- Das Automatenmodell der DFAs wird um einen endlosen Stack erweitert, das ergibt PDAs.
- Kontextfreie Grammatiken (CFGs) erweitern die regulären Grammatiken.
- PDAs akzeptieren kontextfreie Sprachen.
- Deterministisch parsbare Sprachen haben eine eindeutige kontextfreie Grammatik, aber nicht für jede eindeutige kontextfreie Grammatik lässt sich ein deterministischer PDA finden.
- Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.
- Syntaxanalyse wird mit (möglichst deterministisch) kontextfreien Grammatiken durchgeführt.
- In der Praxis werden aus kontextfreien Grammatiken Parser automatisch generiert.

Zum Nachlesen

- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: Ich kenne PDAs
- k1: Ich kenne deterministische PDAs
- k1: Ich kenne kontextfreie Grammatiken
- k1: Ich kenne deterministisch kontextfreie Grammatiken
- k2: Ich kann den Zusammenhang zwischen PDAs und kontextfreien Grammatiken an einem Beispiel erklären

- k2: Ich kann PDAs entwickeln
- k2: Ich kann kontextfreie Grammatiken entwickeln

3.4. LL-Parser

Weitere Unterlagen

- [Annotierte Folien: LL-Parser](#)

3.4.1. Wiederholung

3.4.1.1. PDAs und kontextfreie Grammatiken

- Warum reichen uns DFAs nicht zum Matchen von Eingabezeichen?
- Wie können wir sie minimal erweitern?
- Sind PDAs deterministisch?
- Wie sind kontextfreie Grammatiken definiert?
- Sind kontextfreie Grammatiken eindeutig?

3.4.2. Motivation

3.4.2.1. Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse (=Parser) erstellen lässt
- einen Algorithmus zum Parsen von Programmen mit Hilfe einer solchen Grammatik

3.4.2.2. Themen für heute

- Automatische Generierung von Top-Down-Parsern aus LL-Grammatiken

3.4.3. Syntaxanalyse

3.4.3.1. Syntax

Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

Syntaxanalyse ist die Bestimmung, ob Eingabedaten einer vorgegebenen Syntax entsprechen.

Diese vorgegebene Syntax wird im Compilerbau mit einer kontextfreien Grammatik beschrieben und mit einem sogenannten **Parser** analysiert.

Wir beschäftigen uns heute mit LL-Parsing, mit dem man eine Teilmenge der eindeutigen kontextfreien Grammatiken syntaktisch analysieren kann.

Der Ableitungsbaum wird von oben nach unten aufgebaut.

3.4.3.2. Ziele der Syntaxanalyse

- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- evtl. Fehlerkorrektur
- Bestimmung der syntaktischen Struktur eines Programms
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltabelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, ...), aber auch die Gültigkeitsbereiche.

3.4.4. LL(k)-Grammatiken

3.4.4.1. First-Mengen

$$S \rightarrow A \mid B \mid C$$

Welche Produktion nehmen?

Wir brauchen die “terminalen k-Anfänge” von Ableitungen von Nichtterminalen, um eindeutig die nächste zu benutzende Produktion festzulegen. k ist dabei die Anzahl der Vorschautoken.

Def.: Wir definieren *First* - Mengen einer Grammatik wie folgt:

- $a \in T^*, |a| \leq k : First_k(a) = \{a\}$
- $a \in T^*, |a| > k : First_k(a) = \{v \in T^* \mid a = vw, |v| = k\}$
- $\alpha \in (N \cup T)^* \setminus T^* : First_k(\alpha) = \{v \in T^* \mid \alpha \xRightarrow{*} w, \text{ mit } w \in T^*, First_k(w) = \{v\}\}$

3.4.4.2. Linksableitungen

Def.: Bei einer kontextfreien Grammatik G ist die *Linksableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn in jedem Schritt das am weitesten links stehende Nichtterminal in α abgeleitet wird.

Man schreibt $\alpha \xRightarrow{*}_l \beta$.

3.4.4.3. LL(k)-Grammatiken

Def.: Eine kontextfreie Grammatik $G = (N, T, P, S)$ ist genau dann eine *LL(k)*-Grammatik, wenn für alle Linksableitungen der Form:

$$S \xRightarrow{*}_l wA\gamma \Rightarrow_l w\alpha\gamma \xRightarrow{*}_l wx$$

und

$$S \xRightarrow{*}_l wA\gamma \Rightarrow_l w\beta\gamma \xRightarrow{*}_l wy$$

mit $(w, x, y \in T^*, \alpha, \beta, \gamma \in (N \cup T)^*, A \in N)$ und $First_k(x) = First_k(y)$ gilt:

$$\alpha = \beta$$

3.4.4.4. LL(1)-Grammatiken

Hier entsteht ein Tafelbild.

3.4.4.5. LL(k)-Sprachen

Die von $LL(k)$ -Grammatiken erzeugten Sprachen sind eine echte Teilmenge der deterministisch parsbaren Sprachen.

Die von $LL(k)$ -Grammatiken erzeugten Sprachen sind eine echte Teilmenge der von $LL(k+1)$ -Grammatiken erzeugten Sprachen.

Für eine kontextfreie Grammatik G ist nicht entscheidbar, ob es eine $LL(1)$ -Grammatik G' gibt mit $L(G) = L(G')$.

In der Praxis reichen $LL(1)$ -Grammatiken oft. Hier gibt es effiziente Parsergeneratoren (hier: ANTLR), deren Eingabe eine $LL(k)$ - (meist $LL(1)$ -) Grammatik ist, und die als Ausgabe den Quellcode eines (effizienten) tabellen-gesteuerten Parsers generieren.

3.4.4.6. Algorithmus: Konstruktion einer LL-Parsertabelle

Eingabe: Eine Grammatik $G = (N, T, P, S)$

Ausgabe: Eine Parsertabelle P

```

for each production  $X \rightarrow \alpha$ 
  for each  $a \in First(\alpha)$ 
    add  $X \rightarrow \alpha$  to  $P[X, a]$ 

  if  $\epsilon \in First(\alpha)$ 
    for each  $b \in Follow(\alpha)$ 
      add  $X \rightarrow \alpha$  to  $P[X, b]$ 
      if  $\epsilon \in First(\alpha)$  and  $\perp \in Follow(X)$ 
        add  $X \rightarrow \alpha$  to  $P[X, \perp]$ 

```

Abbildung 3.2.: Algorithmus zur Generierung einer LL-Parsertabelle

Hier ist $\perp$ das Endezeichen des Inputs. Statt $First_1(\alpha)$ wird oft nur $First(\alpha)$ geschrieben.

3.4.4.7. LL-Parsertabellen

Hier entsteht ein Tafelbild.

3.4.4.8. LL-Parsertabellen

Rekursive Programmierung bedeutet, dass das Laufzeitsystem einen Stack benutzt. Diesen Stack kann man auch "selbst programmieren", d. h. einen PDA implementieren. Dabei wird ebenfalls die oben genannte Tabelle zur Bestimmung der nächsten anzuwendenden Produktion benutzt. Der Stack enthält die zu erwartenden Eingabezeichen, wenn immer eine Linksableitung gebildet wird. Diese Zeichen im Stack werden mit dem Input gematcht.

3.4.4.9. Algorithmus: Tabellengesteuertes LL-Parsen mit einem PDA

Eingabe: Eine Grammatik $G = (N, T, P, S)$, eine Parsertabelle P mit " $w \perp$ " als initialem Kellerinhalt

Ausgabe: Wenn $w \in L(G)$, eine Linksableitung von w , Fehler sonst

```

a = next_token()
X = top of stack // entfernt X vom Stack

while X  $\neq$   $\perp$ 
    if X = a
        a = next_token()
    else if X  $\in$  T
        error
    else if P[X, a] leer
        error
    else if P[X, a] =  $X \rightarrow Y_1 Y_2 \dots Y_k$ 
        process_production( $X \rightarrow Y_1 Y_2 \dots Y_k$ )
        push( $Y_1 Y_2 \dots Y_k$ ) //  $Y_1$  = top of stack
    X = top of stack

```

Abbildung 3.3.: Algorithmus zum tabellengesteuerten LL-Parsen

3.4.4.10. Ergebnisse der Syntaxanalyse

- eventuelle Syntaxfehler mit Angabe der Fehlerart und des -Ortes
- Fehlerkorrektur
- Format für die Weiterverarbeitung:
 - Ableitungsbaum oder Syntaxbaum oder Parse Tree
 - abstrakter Syntaxbaum (AST): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. ;, Klammern, manche Schlüsselwörter, ...)

- Symboltabelle

3.4.5. Wrap-Up

3.4.5.1. Wrap-Up

- Syntaxanalyse wird mit deterministisch kontextfreien Grammatiken durchgeführt.
- Eine Teilmenge der dazu gehörigen Sprachen lässt sich top-down parsen.
- Ein effizienter LL(k)-Parser realisiert einen DPDA und kann automatisch aus einer LL(k)-Grammatik generiert werden.
- Der Parser liefert in der Regel einen abstrakten Syntaxbaum.

Zum Nachlesen

- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: Ich kenne die Top-Down-Analyse
- k1: Ich kenne LL-Parser
- k2: Ich kann den algorithmischen Ablauf von LL-Parsern an einem Beispiel erklären

4. Sprache L-Int: Integer, Addition, Subtraktion

4.0.1. Teil 1:

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- AST, InterpreterLInt

4.0.2. Teil 2:

- Wiederholung ANTLR, Visitor mit Rückgabe (zustandslos), ANTLR-Grammatik
- Handgeschriebener Lexer, RD-Parser

5. Sprache L-Expr: erweiterte Ausdrücke, Vorrangregeln

5.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ANTLR: Vorrang mit gestuften Regeln vs. ANTLR4-Precedence
- RD+Pratt
- InterpreterLExpr

6. Sprache L-Var: Variablen, Statments, nested Scopes

6.0.1. Inhalt

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- Nested Scopes, Resolving
- ResolverLVar, InterpreterLVar

7. Sprache L-If: Datentyp Boolean, Vergleiche, Kontrollstrukturen (if/else, while)

7.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ResolverLif, TypeCheckerLif, InterpreterLif
- Nano-Pass mit Desugaring: for, +=, ... (??)

8. Sprache L-Fun: Funktionen (Definition, Aufruf)

8.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ResolverLFun, TypeCheckerLFun, InterpreterLFun
- native Funktionen vs. gepackte Funktionen

9. Sprache L-Class: Klassen (Felder, Methoden, Objekte)

9.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- Klassen als Environment für Methoden, Instanzen als Environment für Felder, Auflöse-Strategien
- ResolverLClass, TypeCheckerLClass, InterpreterLClass

10. Sprache L-Inherit: Einfachvererbung und dynamischer Dispatch

10.0.1. Inhalte

- konkrete Syntax (Grammatik), abstrakte Syntax (AST)
- ResolverLInherit, TypeCheckerLInherit, InterpreterLInherit

11. Sprache L-Self: native Klassen (inkl. Vererbung)

11.0.1. Inhalte

- Einbinden von nativen Klassen
- InterpreterLSelf, TypeCheckerLSelf/ResolverLSelf
- Vorbereitung für Snake

Teil III.

Praktikum

Hier finden Sie die Übungsblätter.

12. Blatt 01: Reguläre Sprachen

12.1. A1.1: Sprachen von regulären Ausdrücken (1P)

Welche Sprache wird von dem folgenden regulären Ausdruck beschrieben?

$$a + a(a + b)^* a$$

12.2. A1.2: Bezeichner in Programmiersprachen (3P)

Betrachten Sie eine Programmiersprache, in der die Bezeichner (= Namen für Variablen, Funktionen, Klassen, Methoden, ...) folgenden Aufbau haben:

- Alle Variablennamen beginnen mit **V** oder **v**
- Handelt es sich um globale Variablen, beginnen Sie mit **V**, lokale beginnen mit **v**
- Funktions- und Methodenparameter beginnen mit **p**, Klassenparameter (bei der Definition von Vererbung) beginnen mit **P**
- Weitere Bezeichner müssen mit einem Buchstaben (a-z, A-Z) beginnen
- Die folgenden Zeichen dürfen Buchstaben, Ziffern und ein Unterstrich sein
- Bezeichner dürfen nicht mit einem Unterstrich enden
- Alle Bezeichner müssen aus mindestens zwei Zeichen bestehen

Entwickeln Sie einen regulären Ausdruck, der den Aufbau der Bezeichner beschreibt. Beachten Sie, dass Ihr regex alle zulässigen Bezeichner beschreiben muss, aber keinen einzigen unzulässigen beschreiben darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie, wie sie vom regex gematcht werden.

Entwickeln Sie einen DFA, der diese Bezeichner akzeptiert. Beachten Sie, dass Ihr DFA alle zulässigen Bezeichner akzeptieren muss, aber keinen einzigen unzulässigen akzeptieren darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie, wie sie vom Automaten zeichenweise gelesen und akzeptiert werden.

Entwickeln Sie eine reguläre Grammatik, die diese Bezeichner generiert. Beachten Sie, dass Ihre Grammatik alle zulässigen Bezeichner generieren können muss, aber keinen einzigen unzulässigen generieren darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie die Ableitungsbäume dazu.

12.3. A1.3: Gleitkommazahlen in Programmiersprachen (2P)

Recherchieren Sie zunächst den Aufbau von Gleitkommazahlen in Python und Java.

Erstellen Sie für jede der beiden Programmiersprachen reguläre Ausdrücke, DFAs und reguläre Grammatiken wie in Aufgabe A1.2. Verifizieren Sie Ihre Lösungen wie in Aufgabe A1.2. Vorgaben, die sich auf Längen oder Werte von Teilen der Zahlen beziehen, ignorieren Sie bitte.

12.4. A1.4: Mailadressen? (1P)

Warum ist der folgende regex ungeeignet für die Verarbeitung von Mailadressen?

$(a - z)^+ @ (a - z) . (a - z)$

Bitte beachten Sie, dass die Schreibweise a-z nicht unserer Definition genügt. Eigentlich müsste jedes Zeichen aufgeführt werden:

$a + b + c + c + \dots + z$ ist besser, aber immer noch nicht richtig. Warum?

Anmerkung: Diese Darstellung wird ab jetzt akzeptiert.

Verbessern Sie den gegebenen regulären Ausdruck.

12.5. A1.5: Der zweitletzte Buchstabe (1P)

Entwickeln Sie einen DFA, der nur Wörter über $\Sigma = \{1, 2, 3\}$ akzeptiert, deren zweitletztes Zeichen dasselbe ist wie das zweite.

12.6. A1.6: Sprache einer regulären Grammatik (2P)

Welche Sprache generiert die folgende Grammatik?

$$S \rightarrow aA$$

$$A \rightarrow dB \mid bA \mid cA$$

$$B \rightarrow aC \mid bC \mid cA$$

$$C \rightarrow \epsilon$$

Können Sie einen regulären Ausdruck oder einen DFA dafür angeben?

13. Blatt 02: CFG

13.1. A2.1: PDA (3P)

Erstellen Sie einen deterministischen PDA, der die Sprache

$$L = \{w \in \{a, b, c\}^* \mid w \text{ hat doppelt so viele a's wie c's}\}$$

akzeptiert.

Beschreiben Sie Schritt für Schritt, wie der PDA die Eingaben *bcaba* und *bccac* abarbeitet.

13.2. A2.2: Akzeptierte Sprache (2P)

Ist der folgenden PDA deterministisch? Warum (nicht)?

q_4 sei der akzeptierende Zustand.

$$\begin{aligned}\delta(q_0, a, \perp) &= (q_0, A \perp) \\ \delta(q_0, a, A) &= (q_0, AA) \\ \delta(q_0, b, A) &= (q_1, BA) \\ \delta(q_1, b, B) &= (q_1, BB) \\ \delta(q_1, c, B) &= (q_2, \epsilon) \\ \delta(q_2, c, B) &= (q_2, \epsilon) \\ \delta(q_2, d, A) &= (q_3, \epsilon) \\ \delta(q_3, d, A) &= (q_3, \epsilon) \\ \delta(q_3, d, A) &= (q_3, AA) \\ \delta(q_3, \epsilon, \perp) &= (q_4, \epsilon)\end{aligned}$$

Zeichnen Sie den Automaten. Geben Sie das 7-Tupel des PDA an. Welche Sprache akzeptiert er?

13.3. A2.3: Kontextfreie Sprache (2P)

Welche Sprache generiert die folgende kontextfreie (Teil-) Grammatik?

$$G = (\{\text{Statement, Condition, } \dots\}, \{\text{"if", "else", } \dots\}, P, \text{Statement})$$

mit

$$P = \left\{ \begin{array}{ll} \text{Statement} & \rightarrow \text{"if" Condition Statement} \mid \text{"if" Condition Statement "else" Statement} \\ \text{Condition} & \rightarrow \dots \end{array} \right\}$$

Ist die Grammatik mehrdeutig? Warum (nicht)?

13.4. A2.4: Kontextfreie Grammatik (3P)

Entwickeln Sie eine kontextfreie Grammatik für die Sprache

$$L = \{a^i b^j c^k \mid i = j \vee j = k\}$$

Zeigen Sie, dass die Grammatik mehrdeutig ist. Entwickeln Sie einen PDA für diese Sprache.

Quellen

Aho, A. V., M. S. Lam, R. Sethi, J. D. Ullman, und S. Bansal. 2023. *Compilers: Principles, Techniques, and Tools, Updated 2nd Edition by Pearson*. Pearson India. <https://learning.oreilly.com/library/view/compilers-principles-techniques/9789357054881/>.

Grune, D., K. van Reeuwijk, H. E. Bal, C. J. H. Jacobs, und K. Langendoen. 2012. *Modern Compiler Design*. Springer.

Hopcroft, J. E., R. Motwani, und J. D. Ullman. 2003. *Einführung in die Automatentheorie, formale Sprachen und Komplexitätstheorie*. I theoretische informatik. Pearson Education Deutschland GmbH.

Parr, T. 2014. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. <https://learning.oreilly.com/library/view/the-definitive-antlr/9781941222621/>.

Torczon, L., und K. Cooper. 2012. *Engineering a Compiler*. Morgan Kaufmann. <https://learning.oreilly.com/library/view/engineering-a-compiler/9780080916613/>.



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- Screenshot of [“Language Brainfuck”](#) by Michal Wojciech Tarnowski on 99-bottles-of-beer.net
- [“Language Prolog”](#) by M@ on 99-bottles-of-beer.net
- [“Language Haskell”](#) by Iavor on 99-bottles-of-beer.net
- Abzählreim “99 Bottles of Beer” nach [“Lyrics of the song 99 Bottles of Beer”](#) on 99-bottles-of-beer.net
- [“Language Java”](#) by Sean Russell on 99-bottles-of-beer.net
- [A Map of the Territory \(mountain.png\)](#) by [Bob Nystrom](#) on Github.com (MIT)
- [“Language C”](#) by Bill Wein on 99-bottles-of-beer.net

Last modified: c51ad86 2026-09-29 orga: update navbar